

Adapt: Diffusion-Predicted Pedestrian Avoidance with MPPI Control on the Polaris GEM e4

CS 588 – Group 10 Mid-Project Report

Polaris GEM e4 Autonomy Team – UIUC CS 588, Spring 2026

May 1, 2026

1. Project Overview

Our project, **Adapt**, builds an autonomous driving stack on the Polaris GEM e4 platform that combines a learning-based pedestrian trajectory predictor with a sampling-based motion planner. Concretely, we replace the default Stanley lateral controller used in the AutoShield baseline with a **Model Predictive Path Integral (MPPI)** planner that reasons jointly over steering and acceleration, and we feed it multi-step pedestrian forecasts produced by a **diffusion trajectory predictor**. The system is implemented in ROS 2 Humble and runs end-to-end on the GEM e4 using its Ouster OS1-128 LiDAR, OAK-D LR RGBD camera, Septentrio GNSS/INS, and PACMod2 drive-by-wire interface.

The core technical contribution is a configurable pipeline with *three* prediction modes (constant-velocity, single-agent diffusion, joint multi-agent diffusion) wired to *two* controllers (MPPI or Stanley), all sharing a common topic contract. This lets us compare a classical baseline against modern generative motion forecasts on the same vehicle, the same routes, and the same safety state machine.

2. System Architecture

The runtime stack is organized in four layers (Fig. 1, conceptual).

Perception. `adapt_lidar_processing` runs DB-SCAN on `/ouster/points` with human-geometry filters (height, width) and publishes polar pedestrian detections on `/lidar_pedestrian_position`. In parallel, `rgbpedestrian_detector` runs YOLOv11 (COCO class 0) on the OAK-D RGB stream and back-projects the bounding-box centroid using the stereo depth image to publish `/rgbpedestrian_position`.

Fusion. `lidar_camera_fusion` performs an approximate time-synced match between LiDAR and camera detections (100 ms slop, 2 m Euclidean gate) and produces a single `/fusion_pedestrian_position` topic at ~ 10 Hz. We weight distance 80/20 in favor of LiDAR (more accurate range) and bearing 30/70 in favor of the camera (better angular resolution).

Prediction. A single `infer_node` consumes the fused pedestrian track and publishes future trajectories on `/pedestrian_predictions_tensor` (an $M \times 20 \times 2$ `Float32MultiArray`), plus legacy `/pedestrian_motion`

and `/pedestrian_ttc` for the high-level decision node. The active prediction model (constant velocity, single-agent diffusion, or joint diffusion) is selected by a launch argument so downstream nodes are mode-agnostic.

Control. The MPPI node subscribes to either `/fusion_pedestrian_position` (raw, static-obstacle treatment) or `/pedestrian_predictions_tensor` (predicted trajectories with per-step covariance) and emits PACMod2 commands at 10 Hz on `/pacmod/steering_cmd`, `/pacmod/accel_cmd`, and `/pacmod/brake_cmd`. A high-level state machine (`adapt_high_level_command`) issues CRUISE / SLOW_CAUTION / STOP_YIELD / CREEP_PASS states based on time-to-collision and pedestrian motion, providing a coarse safety overlay that is independent of the chosen predictor.

3. Diffusion Trajectory Predictor

Both diffusion variants use a MID-style Transformer denoiser trained with DDPM (100 train steps, cosine schedule) and sampled with DDIM-10 at inference. The single-agent model (`TrajectoryDenoiser`, 0.47M parameters) predicts each pedestrian independently from a 20-frame history of $[x, y, v_x, v_y]$ plus ego velocity, using a 4-layer Transformer encoder with 4 attention heads ($d_{model}=128$) and a cross-attention decoder over 20 learned future queries. The joint model (`JointTrajectoryDenoiser`, 0.93M parameters) shares the per-agent encoder, then applies two cross-agent attention layers so each pedestrian’s prediction is conditioned on the trajectories of all others (up to 16 agents, padded with an agent mask).

We pretrain on Argoverse 2 pedestrian splits and fine-tune on rosbags collected from the GEM e4. Current pretrained checkpoints reach **minFDE = 0.693 m** (single-agent) and **minFDE = 0.529 m** (joint) on the held-out AV2 split, with end-to-end inference latency of ~ 9 ms and ~ 11 ms respectively on an RTX 3060. A sticky closest-to-mean selection picks a single trajectory per pedestrian from $K=20$ samples, switching modes only when the candidate beats the current selection by $1.5\times$ for three consecutive ticks; this eliminated TTC jitter we observed when picking the argmax sample each frame.

4. MPPI Motion Planner

The MPPI controller samples $K=600$ control sequences per tick over a horizon of $H=30$ steps at $dt=0.1$ s, integrates them through a kinematic-bicycle model (state (x, y, ψ, v) , input (a, δ)), and returns a softmax-weighted mean. The cost function combines goal-position tracking ($w_{pos}=15$), velocity reference ($w_{vel}=5$), a $|\delta| \cdot v$ stability term ($w_{curv}=2$), a velocity-aware 2D Gaussian repulsion per pedestrian ($w_{obs}=150$), a hard clearance step ($w_{obs,hard}=250$, 3 m radius), and an exponential soft falloff ($w_{obs,soft}=40$). When fed predicted trajectories, the obstacle ellipse *grows* along the rollout horizon, so MPPI is quantitatively more cautious about pedestrians farther in the future. Hard caps from the AutoShield baseline are preserved: $|\delta| \leq 35^\circ$, $v_{max}=5$ m/s, $a_{max}=2$ m/s².

5. Progress to Date

- **Hardware bring-up (complete).** Ouster OS1-128, OAK-D LR, Septentrio GNSS/INS, and PACMod2 are integrated under a single `sensor_init.launch.py` that uses a lifecycle handoff so cameras, GNSS, and RViz only start once the lidar reaches `active`.
- **Baseline controllers (complete).** Pure pursuit and Stanley controllers run end-to-end on the e4 with PID longitudinal control; both gated on the LB+RB joystick enable.
- **MPPI controller (complete, on-vehicle tested).** Torch implementation runs on GPU; standalone Phase-1 test shows mean $|e_{lat}| \approx 0.007$ m on the reference loop and the planner reacts to injected static obstacles within one cycle.
- **Diffusion predictor (complete; fine-tuning in progress).** Both single-agent and joint models are trained on AV2 and integrated as a ROS 2 node; smoothing logic landed this week (commits `4af6828`, `f43e25f`, `d1549dc`) to reduce per-frame mode flips.
- **Sensor fusion + high-level safety (complete).** Fused pedestrian topic at 10 Hz drives both raw-mode

MPPI and the prediction node; high-level node outputs the four safety states identically across all three prediction modes.

6. Remaining Work

- (1) **GEM rosbag fine-tuning.** Pretrained AV2 weights are usable but UIUC pedestrians have a different speed/spacing distribution; we are extracting training windows from on-vehicle rosbags via `extract_gem_windows.py` and will fine-tune both checkpoints before final demos.
- (2) **On-vehicle A/B comparison.** We will run identical routes under (i) Stanley + constant-velocity, (ii) MPPI + constant-velocity, and (iii) MPPI + joint diffusion, logging minimum clearance, intervention rate, and lap time as the primary metrics.
- (3) **Latency budget tightening.** End-to-end perception→control currently sits near 50 ms; we want to push the diffusion-mode worst case under our 80 ms warn threshold by batching DDIM steps on the GPU and avoiding a CPU round-trip in the tracker.
- (4) **Failure-mode analysis.** We need a dedicated set of edge cases (occluded pedestrian behind a parked car, group of pedestrians crossing diagonally, false-positive detections from foliage) to characterize where each predictor wins or loses.

7. Risks and Mitigations

The largest open risk is *prediction overconfidence*: a tight diffusion sample can underestimate variance when the pedestrian is about to change behavior, causing MPPI to plan too aggressively past them. Our confidence-growth obstacle cost partially mitigates this, and the high-level safety node provides a hard `STOP_YIELD` fallback when TTC drops below threshold regardless of which predictor is active. A second risk is sensor degradation in low light or rain; the LiDAR/camera fusion weights deliberately keep LiDAR dominant on range so the system degrades gracefully if the camera channel drops out.